Лабораторная работа №14

Тема работы: «Разработка алгоритмов и программ с использованием списковых структур данных»

1 Цель работы

Сформировать умение реализовывать динамические структуры данных при решении практических задач

2 Задание

1. Разработать программу работы с односвязным кольцом. Программа должна содержать следующие функции, вызываемые из меню:

- построение пустого кольца;
- добавление нового элемента;
- удаление указанного элемента;
- просмотр кольца, начиная с указанного элемента.

12. Используя структуру односвязное кольцо, решить следующую задачу. N ребят встали в круг. Каждый раз, начиная с первого, из круга выводится каждый M-й. Вывести номера в порядке выбывания и номер последнего оставшегося.

14. Разработать программу работы с односвязным списком. Программа должна содержать следующие функции, вызываемые из меню:

- построение пустого списка;
- добавление нового элемента;
- удаление указанного элемента;
- подсчет количества отрицательных элементов списка;
- просмотр списка.

3 Оснащение

ПК, задание по варианту, среда разработки Visual Studio.

4 Порядок выполнения работы

```
1. #include <iostream>
2. #include <ctime>
3. #include <cstdlib>
4. using namespace std;
5.
6. struct Node {
7.     int data;
8.     Node* next;
9. };
10.
11. Node* createEmpty() {
12.     cout << "Пустое кольцо создано" << endl;
13.     return nullptr;
14. }
15.
16. Node* addElement(Node* last, int value) {
17.     Node* newNode = new Node{ value, nullptr };
18.     if (last == nullptr) {
19.         newNode->next = newNode;
20.         return newNode;
21.     }
22.     newNode->next = last->next;
23.     last->next = newNode;
```

```
24.     return newNode;
25. }
26.
27. Node* deleteElement(Node* last, int value) {
28.     if (last == nullptr) {
29.         cout << "Кольцо пустое!" << endl;
30.         return nullptr;
31.     }
32.
33.     if (last->next == last && last->data == value) {
34.         delete last;
35.         cout << "Удален последний элемент" << endl;
36.         return nullptr;
37.     }
38.
39.     Node* curr = last->next;
40.     Node* prev = last;
41.
42.     do {
43.         if (curr->data == value) {
44.             prev->next = curr->next;
45.             if (curr == last) {
46.                 last = prev;
47.             }
48.             delete curr;
49.             cout << "Элемент " << value << " удален" << endl;
50.             return last;
51.         }
52.         prev = curr;
53.         curr = curr->next;
54.     } while (curr != last->next);
55.
56.     cout << "Элемент " << value << " не найден" << endl;
57.     return last;
58. }
59.
60. void viewFromElement(Node* last, int startValue) {
61.     if (last == nullptr) {
62.         cout << "Кольцо пустое!" << endl;
63.         return;
64.     }
65.
66.     Node* curr = last->next;
67.     do {
68.         if (curr->data == startValue) {
69.             cout << "Кольцо: ";
70.             Node* start = curr;
71.             do {
72.                 cout << curr->data << " ";
73.                 curr = curr->next;
74.             } while (curr != start);
75.             cout << endl;
76.             return;
77.         }
78.         curr = curr->next;
79.     } while (curr != last->next);
80.
81.     cout << "Элемент " << startValue << " не найден" << endl;
82. }
83.
84. void clearRing(Node*& last) {
85.     if (last == nullptr) return;
86.
87.     Node* curr = last->next;
88.     while (curr != last) {
89.         Node* temp = curr;
90.         curr = curr->next;
91.         delete temp;
92.     }
93.     delete last;
94.     last = nullptr;
95. }
96.
97. void printCircle(Node* start, int count) {
98.     Node* current = start;
99.     for (int i = 0; i < count; i++) {
```

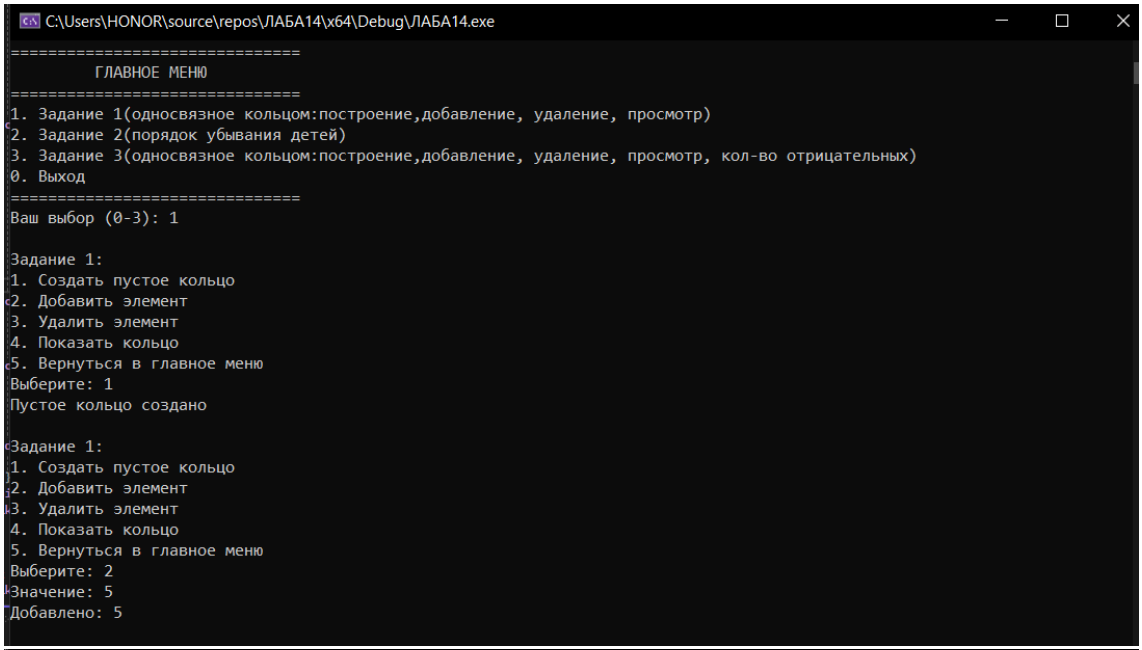
```
        cout << current->data;
        current = current->next;
        if (i < count - 1) cout << " | ";
    }
    cout << endl;
}

void CreatRing(int N, int M) {
    Node* last = new Node{ N, nullptr };
    last->next = last;

    for (int i = N - 1; i > 0; i--) {
        Node* newNode = new Node{ i, last->next };
        last->next = newNode;
    }

    Node* curr = last->next;
    Node* prev = last;

    cout << "Весь круг: ";
    printCircle(curr, N);

    int step = 1;
    while (curr->next != curr) {
        cout << "\nШАГ " << step << endl;
        cout << "Начинаем отсчет с ребенка №" << curr->data << endl;

        for (int i = 1; i < M; i++) {
            prev = curr;
            curr = curr->next;
        }

        cout << "Выбывает ребенок №" << curr->data << endl;
        prev->next = curr->next;

        if (curr == last) {
            last = prev;
        }

        cout << "Оставшийся круг: ";
        printCircle(prev->next, N - step);

        Node* temp = curr;
        curr = curr->next;
        delete temp;
        step++;
    }

    cout << "\nВыбыли по порядку все, кроме:" << endl;
    cout << "Последний оставшийся ребенок №" << curr->data << endl;

    delete curr;
}
int countNegativeElements(Node* last) {
    if (last == nullptr) {
        return 0;
    }

    int count = 0;
    Node* curr = last->next;

    do {
        if (curr->data < 0) {
            count++;
        }
        curr = curr->next;
    } while (curr != last->next);

    return count;
}

int main() {
    setlocale(LC_ALL, "RU");
    srand(time(0));

    int choice;
```

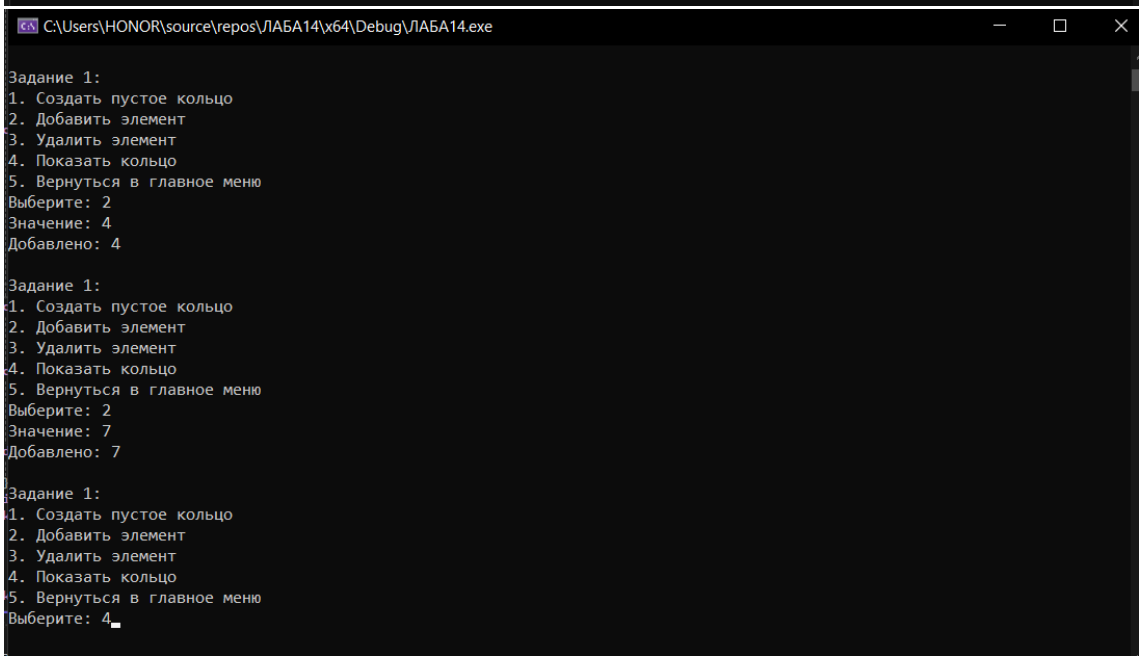
```
176.
177.     do {
178.         cout << "==============================" << endl;
179.         cout << "          ГЛАВНОЕ МЕНЮ          " << endl;
180.         cout << "==============================" << endl;
181.         cout << "1. Задание 1(односвязное кольцом:построение,добавление, удаление, просмотр)" << endl;
182.         cout << "2. Задание 2(порядок убывания детей)" << endl;
183.         cout << "3. Задание 3(односвязное кольцом:построение,добавление, удаление, просмотр, кол-во
отрицательных)" << endl;
184.         cout << "0. Выход" << endl;
185.         cout << "==============================" << endl;
186.         cout << "Ваш выбор (0-3): ";
187.         cin >> choice;
188.
189.         switch (choice) {
190.         case 1: {
191.             Node* last = nullptr;
192.             int menuChoice, value;
193.
194.             do {
195.                 cout << "\nЗадание 1: " << endl;
196.                 cout << "1. Создать пустое кольцо" << endl;
197.                 cout << "2. Добавить элемент" << endl;
198.                 cout << "3. Удалить элемент" << endl;
199.                 cout << "4. Показать кольцо" << endl;
200.                 cout << "5. Вернуться в главное меню" << endl;
201.                 cout << "Выберите: ";
202.                 cin >> menuChoice;
203.
204.                 switch (menuChoice) {
205.                 case 1:
206.                     clearRing(last);
207.                     last = createEmpty();
208.                     break;
209.
210.                 case 2:
211.                     cout << "Значение: ";
212.                     cin >> value;
213.                     last = addElement(last, value);
214.                     cout << "Добавлено: " << value << endl;
215.                     break;
216.
217.                 case 3:
218.                     if (last == nullptr) {
219.                         cout << "Кольцо пустое!" << endl;
220.                     }
221.                     else {
222.                         cout << "Удалить элемент: ";
223.                         cin >> value;
224.                         last = deleteElement(last, value);
225.                     }
226.                     break;
227.
228.                 case 4:
229.                     if (last == nullptr) {
230.                         cout << "Кольцо пустое!" << endl;
231.                     }
232.                     else {
233.                         cout << "Начать с элемента: ";
234.                         cin >> value;
235.                         viewFromElement(last, value);
236.                     }
237.                     break;
238.
239.                 case 5:
240.                     clearRing(last);
241.                     cout << "Возврат в главное меню..." << endl;
242.                     break;
243.
244.                 default:
245.                     cout << "Неверный выбор!" << endl;
246.                 }
247.             } while (menuChoice != 5);
248.             break;
249.         }
250.
```

```
251.         case 2: {
252.             int N, M;
253.
254.             cout << "\nЗадание 2:" << endl;
255.             cout << "Введите количество детей (N): ";
256.             cin >> N;
257.             cout << "Введите шаг для выбывания (M): ";
258.             cin >> M;
259.
260.             CreatRing(N, M);
261.             break;
262.         }
263.
264.         case 3: {
265.             Node* last = nullptr;
266.             int menuChoice, value;
267.
268.             do {
269.                 cout << "\nЗадание 3:" << endl;
270.                 cout << "1. Создать пустое кольцо" << endl;
271.                 cout << "2. Добавить элемент" << endl;
272.                 cout << "3. Удалить элемент" << endl;
273.                 cout << "4. Показать кольцо" << endl;
274.                 cout << "5. Подсчет отрицательных элементов" << endl;
275.                 cout << "6. Вернуться в главное меню" << endl;
276.                 cout << "Выберите: ";
277.                 cin >> menuChoice;
278.
279.                 switch (menuChoice) {
280.                 case 1:
281.                     clearRing(last);
282.                     last = createEmpty();
283.                     break;
284.
285.                 case 2:
286.                     cout << "Значение: ";
287.                     cin >> value;
288.                     last = addElement(last, value);
289.                     cout << "Добавлено: " << value << endl;
290.                     break;
291.
292.                 case 3:
293.                     if (last == nullptr) {
294.                         cout << "Кольцо пустое!" << endl;
295.                     }
296.                     else {
297.                         cout << "Удалить элемент: ";
298.                         cin >> value;
299.                         last = deleteElement(last, value);
300.                     }
301.                     break;
302.
303.                 case 4:
304.                     if (last == nullptr) {
305.                         cout << "Кольцо пустое!" << endl;
306.                     }
307.                     else {
308.                         cout << "Начать с элемента: ";
309.                         cin >> value;
310.                         viewFromElement(last, value);
311.                     }
312.                     break;
313.
314.                 case 5:
315.                     cout << "Количество отрицательных элементов: " << countNegativeElements(last) <<
endl;
316.                     break;
317.
318.                 case 6:
319.                     clearRing(last);
320.                     cout << "Возврат в главное меню..." << endl;
321.                     break;
322.
323.                 default:
324.                     cout << "Неверный выбор!" << endl;
325.                 }
```

```
            } while (menuChoice != 6);
            break;
        }

        case 0:
            cout << "\nВыход из программы..." << endl;
            break;

        default:
            cout << "\nНеверный выбор! Введите число от 0 до 3." << endl;
        }

    } while (choice != 0);

    return 0;
}

```

Результат выполнения работы:





```
Задание 1:
1. Создать пустое кольцо
2. Добавить элемент
3. Удалить элемент
4. Показать кольцо
5. Вернуться в главное меню
Выберите: 4
Начать с элемента: 4
Кольцо: 4 7 5

Задание 1:
1. Создать пустое кольцо
2. Добавить элемент
3. Удалить элемент
4. Показать кольцо
5. Вернуться в главное меню
Выберите: 3
Удалить элемент: 4
Элемент 4 удален

Задание 1:
1. Создать пустое кольцо
2. Добавить элемент
3. Удалить элемент
4. Показать кольцо
5. Вернуться в главное меню
Выберите: ▬
```

Рисунок 1 - Пример выполнения задачи №1(3 скриншота)

```
Ваш выбор (0-3): 2

Задание 2:
Введите количество детей (N): 5
Введите шаг для выбывания (M): 2
Весь круг: 1 | 2 | 3 | 4 | 5

ШАГ 1
Начинаем отсчет с ребенка №1
Выбывает ребенок №2
Оставшийся круг: 3 | 4 | 5 | 1

ШАГ 2
Начинаем отсчет с ребенка №3
Выбывает ребенок №4
Оставшийся круг: 5 | 1 | 3

ШАГ 3
Начинаем отсчет с ребенка №5
Выбывает ребенок №1
Оставшийся круг: 3 | 5

ШАГ 4
Начинаем отсчет с ребенка №3
Выбывает ребенок №5
Оставшийся круг: 3
```

Рисунок 2 - Пример выполнения задачи №2

```
Задание 3:
1. Создать пустое кольцо
2. Добавить элемент
3. Удалить элемент
4. Показать кольцо
5. Подсчет отрицательных элементов
6. Вернуться в главное меню
Выберите: 4
Начать с элемента: -9
Кольцо: -9 5 -7

Задание 3:
1. Создать пустое кольцо
2. Добавить элемент
3. Удалить элемент
4. Показать кольцо
5. Подсчет отрицательных элементов
6. Вернуться в главное меню
Выберите: 5
Количество отрицательных элементов: 2
```

Рисунок 3 - Пример выполнение задачи №3

Контрольные вопросы:

1. Дайте краткое понятие списковых структур данных.

Списковые структуры данных — это динамические структуры, состоящие из узлов, связанных между собой указателями, позволяющие эффективно управлять памятью и элементами.

2. Раскройте значение односвязного списка и опишите его структуру с помощью узлов.

Односвязный список — это последовательность узлов, где каждый узел хранит данные и указатель на следующий узел.

3. Сформулируйте сущность двусвязного списка и его отличия от односвязного. Какие действия можно выполнять с двусвязным списком, но не с односвязным?

Двусвязный список — это структура, где каждый узел содержит ссылки на предыдущий и следующий узлы. В отличие от односвязного, позволяет двигаться в обе стороны и удалять узлы без прохода от начала списка.

4. Изложите особенности использования циклического списка в программировании и как это отличается от обычного списка.

Циклический список — это список, где последний узел указывает на первый. Отличие от обычного: можно организовать бесконечный обход элементов, удобно для реализации очередей и кольцевых буферов.

5. Дайте краткое понятие указателя и объясните его роль в работе со списковыми структурами данных на языке C++.

Указатель — это переменная, хранящая адрес другой переменной. В списках C++ указатели связывают узлы между собой и обеспечивают динамическое управление памятью.

6. Сформулируйте сущность структуры узла (Node) для односвязного списка и какие данные обычно хранятся в узле списка.

Узел (Node) — элемент списка, содержащий поле данных и указатель на следующий узел.

7. Изложите особенности добавления элемента в начало и конец односвязного списка и как это влияет на сложность алгоритма.

Добавление в начало выполняется за $O(1)$, так как меняется только указатель головы. Добавление в конец требует прохода по списку и занимает $O(n)$.

8. Раскройте особенности удаления элемента из середины двусвязного списка и как это может повлиять на производительность программы.

В двусвязном списке удаление из середины выполняется за $O(1)$, если известен указатель на узел, так как можно напрямую изменить ссылки соседних узлов. Это повышает производительность по сравнению с односвязным списком.

9. Дайте определение структуры данных "стек" и объясните, как он реализуется на основе списка. Какие операции можно выполнять со стеком на основе списка?

Стек — это структура данных типа LIFO (последним вошёл — первым вышел).